Image Comparison Slider (Before / After)

Problem: Build a before/after image comparison slider: two images stacked, a draggable divider reveals more of one or the other.
Often asked at: Adobe, Myntra · **Tests:** Pointer events, clip-path
Tech stack: React (Vite, JavaScript, plain CSS) · **Time limit:** 60 minutes

---

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what it must do, what is out of scope. Write it down.
2. **List interactions**: every action the user can take and the expected result. This is your test checklist.
3. **Sketch the architecture**: components and who owns what.
4. **Design state**: the smallest set of values; derive everything else.
5. **Define contracts**: props in, events out, data shapes.
6. **Trace one flow end to end** before coding.
7. **Name the hard parts** and decide their solution up front.
8. **Build in layers**: a static version first, then one working layer at a time. Always have something that runs.
9. **Cover states and edge cases**: loading, error, empty, extremes.
10. **Wrap up**: trade-offs, what you would add with more time.

Time budget (60 minutes)

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state	7 min
Static layout: two stacked images + divider (CSS only)	8 min
Drag with pointer events	15 min
Keyboard + accessibility	8 min
Loading / error states, edge cases	9 min
Polish, demo, trade-offs	8 min

Think out loud

Say every decision and its reason as you make it: "I'll store the position as a percent, not pixels, so a resize needs no recalculation." The interviewer grades the reasoning as much as the result. If you are stuck, say what you are trying and why.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show two images of the same scene in the same box: **before** on the left side of the divider, **after** on the right.
- A vertical **divider with a handle** sits at a position between 0% and 100%.
- **Drag** the handle (mouse, touch, pen) to move the divider; the visible share of each image updates live.
- **Click / tap** anywhere on the image to jump the divider there.
- **Keyboard**: focus the handle, arrows move it, Shift + arrow / PageUp / PageDown move faster, Home / End go to the edges.
- Labels "Before" and "After".
- Configurable starting position; reports position changes to the parent.

Non-functional requirements

Area	What it means here
Performance	Smooth at 60-120 Hz pointer moves; only a CSS variable changes per move, no layout of the images
Responsiveness	Works at any width; position survives resizes (store percent, not px)
Error handling	Image fails to load -> clear message, no broken slider
Accessibility	`role="slider"` handle with `aria-valuenow` , visible focus ring, full keyboard control, alt text on both images
Usability	Touch: horizontal drag moves the slider, vertical swipe still scrolls the page; no image ghost-drag, no text selection
Reusability	One `<ImageCompare>` component driven by props, no global state
Extensibility	Vertical orientation, controlled mode, hover mode can be added without a rewrite

Clarifying questions to ask

1. Are both images the **same size / aspect ratio**? If not, crop (cover) or letterbox (contain)?
2. Do we need **horizontal only**, or also a vertical (top/bottom) split?
3. Should **clicking** the image jump the divider, or only dragging the handle moves it?
4. Do we need **keyboard and screen reader** support? (Say you will add it anyway.)
5. Should it move on **hover** (no click) like some product pages, or only on drag?
6. Is the position **controlled** by the parent (e.g. synced with another UI), or internal?

Assumptions we build with

- Horizontal split only; before on the left, after on the right.

- Images may differ in size: we fix the frame's aspect ratio (prop, default `3 / 2`) and use `object-fit: cover` for both.
- Drag the handle **or** click anywhere on the image to jump.
- Uncontrolled component with `initial` prop and `onChange` callback.
- Full keyboard + ARIA support.
- Image URLs come in as props; no API.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Placeholder "Loading images..." in a box of the final size (no layout shift)
Both images loaded	Images show, divider + handle at `initial` (50%)
An image fails to load	"Couldn't load the images." message, no handle
Pointer down on the image (left button / touch / pen)	Divider jumps to the pointer x
Drag (pointer moves while pressed)	Divider follows the pointer; before image is clipped at the divider
Drag out of the frame / out of the window	Keeps tracking; position clamps at 0% / 100%
Release (pointer up / cancel)	Dragging stops
Vertical swipe on touch	Page scrolls, slider does not move
Right click / second finger	Ignored
Tab to the handle	Focus ring shows
ArrowLeft / ArrowDown	-1% (or `step`)
ArrowRight / ArrowUp	+1%
Shift + arrow, PageUp / PageDown	±10%
Home / End	0% / 100%
Window resize	Divider keeps the same percent, so it stays over the same part of the picture
Any position change	`onChange(percent)` is called

---

3. Design the architecture

Piece	Responsibility
`App`	Demo page: passes image URLs, shows the current percent
`ImageCompare`	Owns position + load state; handles pointer and keyboard; renders the layers

Piece	Responsibility
`utils/position.js`	Pure math: clamp, pointer x -> percent, key -> next percent
`ImageCompare.css`	Stacking, clip-path reveal, divider, handle, states

```
App
└── ImageCompare  (state: pos, loadedCount, hasError)
    └── .ic-frame  (pointer handlers, --pos / --ratio CSS vars)
        ├── .ic-layer           after image + "After" label   (full)
        ├── .ic-layer.ic-before before image + "Before" label (clip-path at --pos)
        ├── .ic-divider         line at left: --pos
        ├── .ic-handle          role="slider", keyboard
        └── .ic-status          loading / error overlay
```

Start simple: one component plus a tiny pure util file. Divider and handle are a few lines of JSX each, so they stay inline; split them out only if they grow (custom handle render prop, vertical mode).

---

4. Design the state / data model

State or field	Purpose
`pos` (number 0-100)	Divider position in **percent** of the frame width
`loadedCount` (0-2)	How many images fired `onLoad`
`hasError` (boolean)	Any image fired `onError`
`frameRef` (ref)	The frame DOM node, to read its rect for pointer math

```
const [pos, setPos] = useState(() => clamp(initial));
const [loadedCount, setLoadedCount] = useState(0);
const [hasError, setHasError] = useState(false);
const frameRef = useRef(null);

const ready = loadedCount >= 2 && !hasError; // derived
```

Senior note: derive, don't store

- **Pixels are derived**: CSS turns `--pos: 37%` into the divider's `left` and the clip inset. No px state, no resize listener.
- `ready` **is derived** from `loadedCount` and `hasError`.
- **"Is dragging" is not state**: the browser already knows. `setPointerCapture` on pointer down and `hasPointerCapture` on move; capture is released automatically on pointer up / cancel. No stale-closure bugs, no extra render.
- **Clip value, aria values, label text**: all computed from `pos` in render.
- **Ownership**: `ImageCompare` owns `pos` (uncontrolled). The parent only listens via `onChange`. If the parent needs to set it, switch to controlled (`value` + `onChange`), see Q8.

---

5. Design the data structures and contracts

Example data

```
// Same photo twice so the comparison is meaningful: grayscale vs color.
const BEFORE = 'https://picsum.photos/id/1018/1200/800?grayscale';
const AFTER  = 'https://picsum.photos/id/1018/1200/800';
```

Why: same scene + same size proves the alignment is exact; a visible difference (gray vs color) makes the reveal obvious in a demo.

Component contract

```
<ImageCompare
  beforeSrc="string"        // required
  afterSrc="string"         // required
  beforeAlt="string"        // default "Before"
  afterAlt="string"         // default "After"
  initial={50}              // starting percent, clamped to 0-100
  step={1}                  // keyboard step in percent
  ratio="3 / 2"             // frame aspect ratio (CSS aspect-ratio value)
  onChange={(pct) => {}}    // called with a number 0-100 on every change
/>
```

Util contracts (pure, easy to test)

```
clamp(n, min = 0, max = 100)              // -> number
percentFromPointer(clientX, rect)         // -> 0..100   (rect from getBoundingClientRect)
nextFromKey(key, pos, step = 1)           // -> 0..100 or null if the key is not handled

percentFromPointer(250, { left: 100, width: 300 }) // 50
percentFromPointer(-40, { left: 100, width: 300 }) // 0   (clamped)
nextFromKey('End', 37)                             // 100
nextFromKey('a', 37)                               // null
```

No API: images are plain URLs. A real app would get them from a CMS / product API, e.g. `{ id: 'p1', before: { url, alt }, after: { url, alt } }`.

---

6. Trace the data flow

Drag the handle from 50% to 70%:

```
pointerdown on .ic-frame (clientX = 450)
        │  isPrimary && left button && ready ?
        ▼
setPointerCapture(pointerId)  ── browser now sends every move/up to the frame
        │
        ▼
percentFromPointer(450, frame.getBoundingClientRect())  -> 50
```

```
        |
        ▼
setPos(50) + onChange(50)
        |
pointermove (clientX = 530)  ── hasPointerCapture ? yes
        ▼
percentFromPointer(530, rect) -> 70  ->  setPos(70) + onChange(70)
        |
        ▼
React re-renders: style="--pos: 70%"
        |
        ▼
CSS: .ic-before  clip-path: inset(0 calc(100% - 70%) 0 0)
     .ic-divider / .ic-handle  left: 70%
     aria-valuenow = 70
        |
pointerup  ── capture released automatically, next moves are ignored
```

---

7. Important technical challenges

Challenge 1: Revealing part of an image without distorting it

Problem: The naive way is a wrapper with `width: 70%; overflow: hidden` around the before image. But the image inside is `width: 100%` of the **wrapper**, so it shrinks as you drag: at 30% the before photo is squeezed to 30% width and no longer lines up with the after photo.

Solution: Stack both images at full size in the same box and **clip** the top one. `clip-path: inset(top right bottom left)` hides everything outside the inset box without changing layout. Cutting `100% - pos` from the right shows exactly `pos` percent from the left. Drive it with one CSS variable.

```
.ic-layer  { position: absolute; inset: 0; }
.ic-img    { width: 100%; height: 100%; object-fit: cover; }
.ic-before { clip-path: inset(0 calc(100% - var(--pos)) 0 0); }
.ic-divider, .ic-handle { left: var(--pos); }
```

```
<div className="ic-frame" style={{ '--pos': `${pos}%` }}> ... </div>
```

Bonus: putting the "Before" label **inside** the clipped layer means it disappears naturally when the divider passes over it.

Challenge 2: Dragging that works for mouse, touch and pen, even outside the frame

Problem: With `mousemove` on the element, a fast drag leaves the frame and the slider stops. Adding `mousemove` on `window` means manual add/remove of listeners, cleanup on unmount, and a separate set of `touchstart/touchmove` handlers. On phones the browser also starts scrolling or ghost-dragging the image instead of moving the slider.

Solution:

- **Pointer events**: one API for mouse, touch and pen.

- `setPointerCapture`: after pointer down, all moves and the up event go to the frame, even when the pointer is outside it or outside the window. Capture is released on up / cancel, so `hasPointerCapture` is the drag flag.
- `touch-action: pan-y`: the browser keeps vertical scrolling; horizontal moves come to us.
- `draggable={false}` + `pointer-events: none` on images + `user-select: none`: no native image drag, no text selection.
- `e.isPrimary` and `e.button === 0`: ignore second fingers and right clicks.

```
const handlePointerDown = (e) => {
  if (!ready || !e.isPrimary || e.button !== 0) return;
  e.currentTarget.setPointerCapture(e.pointerId);
  moveTo(e.clientX);
};

const handlePointerMove = (e) => {
  if (e.currentTarget.hasPointerCapture(e.pointerId)) moveTo(e.clientX);
};
```

Challenge 3: Pointer x -> percent, correct with scroll, resize and edges

Problem: `e.offsetX` is relative to whatever child is under the pointer (the handle, the label), so it jumps. `pageX - offsetLeft` breaks inside scrolled or transformed containers. Values outside the frame give -20% or 130%.

Solution: Use viewport coordinates on both sides: `clientX` and `getBoundingClientRect()` (both relative to the viewport, both already include scroll and transforms). Read the rect on every move, so a resize or scroll mid-drag is still correct. Clamp to 0-100. Guard `width === 0` (hidden tab, `display: none`).

```
export function percentFromPointer(clientX, rect) {
  if (!rect.width) return 50;
  return clamp(((clientX - rect.left) / rect.width) * 100);
}
```

Challenge 4: Keyboard and screen reader access

Problem: A drag-only widget is unusable without a mouse, and a screen reader announces nothing.

Solution: Make the handle a focusable `role="slider"` with `aria-valuemin/max/now` and a readable `aria-valuetext`. Map keys in a pure function; `preventDefault` only for keys we handle so Tab still works and arrows don't scroll the page.

```
const handleKeyDown = (e) => {
  const next = nextFromKey(e.key, pos, e.shiftKey ? 10 : step);
  if (next === null) return;
  e.preventDefault();
  update(next);
};

<div role="slider" tabIndex={0}
  aria-label="Before and after comparison"
  aria-valuemin={0} aria-valuemax={100} aria-valuenow={rounded}
```

```
  aria-valuetext={`${rounded}% of the before image visible`}
  onKeyDown={handleKeyDown} />
```

8. Implement in the correct order

#	Step	Working result
1	`App` renders `ImageCompare` with two URLs	Two images on the page
2	Frame with `aspect-ratio`, both layers `position: absolute; inset: 0`, `object-fit: cover`	Images stacked exactly on top of each other
3	Hard-code `--pos: 50%` and the `clip-path` on the before layer	Half before, half after (CSS only, no JS yet)
4	Divider + handle at `left: var(--pos)`	Static slider look
5	`pos` state, write it to `--pos`	Changing `initial` moves everything
6	`percentFromPointer` + `onPointerDown` (jump on click)	Click to move
7	`setPointerCapture` + `onPointerMove`	Drag works, also outside the frame
8	`touch-action: pan-y`, `draggable={false}`, `user-select: none`, `isPrimary` check	Clean touch and mouse behavior
9	Handle `role="slider"` + `nextFromKey` + focus ring	Keyboard + screen reader
10	`loadedCount` / `hasError` + status overlay	Loading and error states
11	Labels inside layers, `onChange` callback, demo readout	Feature complete
12	Edge-case pass (0/100, resize, right click, hidden frame)	Done

9. Cover all states and edge cases

State / edge case	Handling
Initial (images loading)	Frame already has its final size via `aspect-ratio` (no layout shift); layers hidden; "Loading images…"
Success (both loaded)	Layers visible, divider + handle rendered at `initial`
Error (any image fails)	`role="alert"` message; handle not rendered; pointer down ignored
Empty (missing `src`)	`img` fires `onError` -> error state
`initial` out of range (-10, 150)	Clamped to 0 / 100 at init
Position 0%	Before fully hidden (`inset(0 100% 0 0)`), handle at left edge

State / edge case	Handling
Position 100%	Before fully visible
Drag beyond the frame / window	Pointer capture keeps events; value clamped
Window resize / zoom	Percent state, so the divider stays on the same part of the image
Frame hidden (`width: 0`)	`percentFromPointer` returns 50 instead of `NaN` / `Infinity`
Images of different sizes	Both `object-fit: cover` in the same box, so they stay aligned
Right click / middle click	Ignored (`e.button !== 0`)
Second finger (pinch)	Ignored (`!e.isPrimary`)
Vertical swipe on mobile	Page scrolls (`touch-action: pan-y`)
Pointer cancel (OS gesture, alert)	Capture released, drag stops
Native image drag / text selection	Disabled
Keyboard keys we don't handle (Tab)	Not prevented
`src` changes to a new pair	Parent passes `key={afterSrc}` so load state resets (see Q10)

10. Final structure

```
_88_image_comparison_slider/
├── index.html
├── package.json
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── App.css
    ├── components/
    │   ├── ImageCompare.jsx
    │   └── ImageCompare.css
    └── utils/
        └── position.js
```

11. Full working code

`src/utils/position.js`

```
export const clamp = (n, min = 0, max = 100) => Math.min(max, Math.max(min, n));

// Pointer x (viewport px) -> slider position (0-100 %) inside the frame.
```

```
export function percentFromPointer(clientX, rect) {
  if (!rect.width) return 50; // hidden / not laid out yet
  return clamp(((clientX - rect.left) / rect.width) * 100);
}

// Keyboard key -> next position, or null when the key is not ours.
export function nextFromKey(key, pos, step = 1) {
  switch (key) {
    case 'ArrowLeft':
    case 'ArrowDown':
      return clamp(pos - step);
    case 'ArrowRight':
    case 'ArrowUp':
      return clamp(pos + step);
    case 'PageDown':
      return clamp(pos - 10);
    case 'PageUp':
      return clamp(pos + 10);
    case 'Home':
      return 0;
    case 'End':
      return 100;
    default:
      return null;
  }
}
```

`src/components/ImageCompare.jsx`

```
import { useRef, useState } from 'react';
import { clamp, percentFromPointer, nextFromKey } from '../utils/position';
import './ImageCompare.css';

export default function ImageCompare({
  beforeSrc,
  afterSrc,
  beforeAlt = 'Before',
  afterAlt = 'After',
  initial = 50,
  step = 1,
  ratio = '3 / 2',
  onChange,
}) {
  const [pos, setPos] = useState(() => clamp(initial));
  const [loadedCount, setLoadedCount] = useState(0);
  const [hasError, setHasError] = useState(false);
  const frameRef = useRef(null);

  const ready = loadedCount >= 2 && !hasError;
  const rounded = Math.round(pos);

  const update = (next) => {
    setPos(next);
    onChange?.(next);
  };

  const moveTo = (clientX) => {
    update(percentFromPointer(clientX, frameRef.current.getBoundingClientRect()));
```

```
  };

  const handlePointerDown = (e) => {
    if (!ready || !e.isPrimary || e.button !== 0) return;
    // Capture: keep getting moves even when the pointer leaves the frame.
    e.currentTarget.setPointerCapture(e.pointerId);
    moveTo(e.clientX); // click anywhere = jump there
  };

  const handlePointerMove = (e) => {
    // Pointer capture is the "is dragging" flag; it is released on up/cancel.
    if (e.currentTarget.hasPointerCapture(e.pointerId)) moveTo(e.clientX);
  };

  const handleKeyDown = (e) => {
    const next = nextFromKey(e.key, pos, e.shiftKey ? 10 : step);
    if (next === null) return;
    e.preventDefault(); // stop arrows / PageUp from scrolling the page
    update(next);
  };

  const handleLoad = () => setLoadedCount((n) => n + 1);
  const handleError = () => setHasError(true);

  return (
    <figure className="ic">
      <div
        ref={frameRef}
        className={`ic-frame${ready ? '' : ' is-loading'}`}
        style={{ '--pos': `${pos}%`, '--ratio': ratio }}
        onPointerDown={handlePointerDown}
        onPointerMove={handlePointerMove}
      >
        <div className="ic-layer">
          <img className="ic-img" src={afterSrc} alt={afterAlt} draggable={false} onLoad={handleLoad}
onError={handleError} />
          <span className="ic-label ic-label-after">After</span>
        </div>

        <div className="ic-layer ic-before">
          <img className="ic-img" src={beforeSrc} alt={beforeAlt} draggable={false} onLoad=
{handleLoad} onError={handleError} />
          <span className="ic-label ic-label-before">Before</span>
        </div>

        {ready && (
          <>
            <div className="ic-divider" aria-hidden="true" />
            <div
              className="ic-handle"
              role="slider"
              tabIndex={0}
              aria-label="Before and after comparison"
              aria-valuemin={0}
              aria-valuemax={100}
              aria-valuenow={rounded}
              aria-valuetext={`${rounded}% of the before image visible`}
              onKeyDown={handleKeyDown}
            >
              <span aria-hidden="true">&#x2039;&#x203A;</span>
```

```
            </div>
          </>
        )}

        {!ready && !hasError && <div className="ic-status">Loading images…</div>}
        {hasError && (
          <div className="ic-status" role="alert">
            Couldn't load the images.
          </div>
        )}
      </div>
    </figure>
  );
}
```

`src/components/ImageCompare.css`

```
.ic {
  margin: 0;
  width: 100%;
  max-width: 900px;
}

.ic-frame {
  position: relative;
  overflow: hidden;
  aspect-ratio: var(--ratio);
  border-radius: 12px;
  background: #e5e7eb;
  cursor: ew-resize;
  touch-action: pan-y; /* horizontal drags are ours, vertical swipes still scroll */
  user-select: none;
  -webkit-user-select: none;
}

.ic-layer {
  position: absolute;
  inset: 0;
}

.ic-img {
  width: 100%;
  height: 100%;
  object-fit: cover; /* both images fill the same box even if sizes differ */
  display: block;
  pointer-events: none;
}

.ic-before {
  clip-path: inset(0 calc(100% - var(--pos)) 0 0);
}

.is-loading .ic-layer {
  opacity: 0;
}

.ic-label {
  position: absolute;
```

```
  top: 12px;
  padding: 4px 8px;
  border-radius: 4px;
  background: rgb(0 0 0 / 0.6);
  color: #fff;
  font-size: 12px;
  font-weight: 600;
}

.ic-label-before {
  left: 12px;
}

.ic-label-after {
  right: 12px;
}

.ic-divider {
  position: absolute;
  top: 0;
  bottom: 0;
  left: var(--pos);
  width: 2px;
  background: #fff;
  transform: translateX(-50%);
  pointer-events: none;
}

.ic-handle {
  position: absolute;
  top: 50%;
  left: var(--pos);
  width: 44px;
  height: 44px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: #fff;
  color: #111827;
  font-size: 20px;
  box-shadow: 0 2px 8px rgb(0 0 0 / 0.35);
  transform: translate(-50%, -50%);
}

.ic-handle:focus-visible {
  outline: 3px solid #2563eb;
  outline-offset: 2px;
}

.ic-status {
  position: absolute;
  inset: 0;
  display: grid;
  place-items: center;
  color: #374151;
  font-size: 14px;
}
```

`src/App.jsx`

```jsx
import { useState } from 'react';
import ImageCompare from './components/ImageCompare';
import './App.css';

// Same photo twice: grayscale = "before", color = "after".
const BEFORE = 'https://picsum.photos/id/1018/1200/800?grayscale';
const AFTER = 'https://picsum.photos/id/1018/1200/800';

export default function App() {
  const [pos, setPos] = useState(50);

  return (
    <main className="app">
      <h1>Image comparison slider</h1>
      <ImageCompare
        beforeSrc={BEFORE}
        afterSrc={AFTER}
        beforeAlt="Mountain lake in black and white"
        afterAlt="Mountain lake in color"
        onChange={setPos}
      />
      <p className="app-hint">
        Before visible: {Math.round(pos)}% · drag, click, or focus the handle and use the arrow keys
      </p>
    </main>
  );
}
```

`src/App.css`

```css
.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px;
  padding: 32px 16px;
  font-family: system-ui, sans-serif;
  background: #f9fafb;
}

.app h1 {
  margin: 0;
  font-size: 24px;
}

.app-hint {
  margin: 0;
  color: #4b5563;
  font-size: 14px;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

---

12. Wrap-up

Edge cases handled

- Drag outside the frame and window (pointer capture), clamped 0-100.
- Mouse, touch and pen with one code path; vertical page scroll kept on phones.
- Right click, second finger, native image drag, text selection ignored.
- Resize-safe (percent state), zero-width frame guarded.
- Loading without layout shift, error state, images of different sizes.
- Full keyboard + `role="slider"` for screen readers.

Trade-offs to mention

Decision	Why	Cost
`clip-path` reveal	No image distortion, one CSS variable, GPU-friendly	Very old browsers lack `clip-path` (fallback: width wrapper with a fixed-width inner image)
Percent state	Resize for free	Needs the rect on every move (cheap, one read per event)
Pointer capture as drag flag	No extra state, no window listeners, no cleanup	Must use `currentTarget` (the frame) consistently
React state per move	Simple, readable, `onChange` comes for free	One render per pointer event; fine for one small component, see Q6 for the ref + rAF version
Fixed `aspect-ratio` + `cover`	No layout shift, aligned images	Edges may be cropped if the images' ratio differs from the frame
Click anywhere jumps	Faster on touch, matches most implementations	Grabbing the handle slightly off-centre jumps by a few px (see Q17)

With more time

- **Tests**: unit tests for `clamp`, `percentFromPointer`, `nextFromKey`; React Testing Library for keyboard + aria values; Playwright for real drag and touch.
- **Controlled mode** (`value` prop), **vertical** orientation, **hover** mode.

- **rAF-throttled updates via a ref** for pages with many sliders.
- `srcSet` / `sizes`, `loading="lazy"`, `decoding="async"` for big photos.
- RTL support, `prefers-reduced-motion` for any animated jump.

---

13. Counter questions the interviewer may ask

Requirements

> **Q1. Why did you ask if both images have the same size?**

Because alignment is the whole point of the widget. If sizes differ and you just stack them, the "before" tree sits in a different spot than the "after" tree. Fixing the frame's `aspect-ratio` and using `object-fit: cover` on both makes them occupy the identical box; `contain` is the choice if cropping is not allowed.

> **Q2. What if the interviewer wants it to move on hover, without clicking?**

Remove the capture check: on `pointermove` with `pointerType === 'mouse'`, always call `moveTo`. Keep drag for touch, since touch has no hover. Make it a `mode="hover" | "drag"` prop.

> **Q3. What if they ask for a vertical (top / bottom) split?**

Same component with an `orientation` prop. Use `clientY`, `rect.top` and `rect.height` in the math, `clip-path: inset(0 0 calc(100% - var(--pos)) 0)`, `top: var(--pos)` for the divider, `touch-action: pan-x`, `aria-orientation="vertical"`, and ArrowUp/Down mapping.

State design

> **Q4. Why store a percent and not pixels?**

Pixels depend on the current width. Store px and every resize, zoom or rotate puts the divider over a different part of the picture unless you add a ResizeObserver to recompute. A percent means "this point of the image" at any size, and CSS turns it into px for free.

> **Q5. Why no `isDragging` state?**

The browser already tracks it: after `setPointerCapture`, `hasPointerCapture(pointerId)` is true until pointer up or cancel, when the browser releases it. A separate state could get out of sync (missed `pointerup` outside the window) and causes an extra render. Add state only if you need it for styling (e.g. a "grabbing" cursor).

> **Q6. A `setState` on every `pointermove` at 120 Hz: is that a performance problem?**

For one small component, no: the render only changes a CSS variable and two attributes, and React batches updates. If profiling shows jank (many sliders, heavy parent), write `--pos` directly via `frameRef.current.style.setProperty('--pos', ...)` inside a `requestAnimationFrame`, keep the value in a ref, and commit to state / `onChange` on pointer up. That is at most one style write per frame and zero renders while dragging.

Q7. Why a CSS variable instead of setting `left` and `clip-path` inline on each element?

One value, written once, read by three rules (clip, divider, handle). It keeps the geometry in CSS where it belongs, and it is the hook that makes the Q6 optimisation (write one style property without React) a one-liner.

Q8. How would you make it controlled?

Accept `value` and `onChange`. If `value` is defined use it, else use internal state: `const pos = value ?? internalPos`. Every change calls `onChange`, and only sets internal state when uncontrolled. This is the same pattern as `<input value>` vs `defaultValue`; `initial` plays the role of `defaultValue`.

Hard parts

Q9. Why `clip-path` and not a wrapper with `width: X%; overflow: hidden`?

A wrapper changes the box the image is sized against, so the image shrinks with the wrapper and stops lining up. You can fix that by giving the inner image a fixed width equal to the frame (needs a measured px width and a resize listener). `clip-path` hides pixels without touching layout, so both images keep identical boxes with no measuring. An alternative is two `transform: translateX` layers moving in opposite directions, but that is more code for the same result.

Q10. What if the image is slow or fails to load?

The frame has its final size from `aspect-ratio`, so nothing jumps; a "Loading…" overlay shows until both `onLoad` events fire. If either fires `onError`, an alert replaces the slider. When the `src` pair changes, the parent passes `key={afterSrc}` so React remounts and the load count resets; inside the component you could instead track loaded URLs in a `Set`.

Q11. Why pointer events and not mouse + touch events?

One code path for mouse, touch and pen, plus `setPointerCapture`, `isPrimary` and `pointercancel`, which the mouse/touch pair doesn't give you cleanly. With mouse + touch you also get the "ghost" emulated mouse events after a tap and have to `preventDefault` them.

Q12. Why `setPointerCapture` and not `window.addEventListener('pointermove')`?

Window listeners need manual add on down, remove on up, remove on unmount, and they can leak if `pointerup` happens outside the window. Capture routes all events of that pointer to the element and is released automatically. It is also per pointer, so a second finger doesn't hijack the drag.

Q13. What does `touch-action: pan-y` do and what breaks without it?

It tells the browser it may handle vertical panning itself, but horizontal moves belong to the page script. Without it, on a phone the browser may start a scroll and fire `pointercancel` mid-drag, so the slider stops after a few pixels. With `touch-action: none` the slider works but the user can't scroll the page when their finger lands on the image, which is bad on a product page.

Q14. Why `getBoundingClientRect` + `clientX` and not `offsetX` or `pageX`?

`offsetX` is relative to the element under the pointer: over the handle or a label it suddenly jumps. `pageX` needs the element's page offset, which is wrong inside scrolled containers or with CSS transforms. `clientX` and the rect are both in viewport coordinates and include scroll and transforms.

Q15. Isn't calling `getBoundingClientRect` on every move expensive?

It can force layout if styles changed since the last frame. Here the only change is a CSS variable that affects paint, not the frame's geometry, so the read is cheap. If needed, cache the rect on pointer down and refresh it on `scroll` / `resize` during the drag.

Q16. Why `role="slider"` on a div, not a native `<input type="range">`?

A native range gives keyboard and aria for free, and is a valid choice: stretch an invisible range over the frame and style the handle yourself. The cost is fighting browser styles for the thumb and getting click-anywhere + touch-action right on the input. The custom slider is more code but fully under our control; mention both and pick based on time.

Implementation steps

Q17. When I grab the handle slightly off-centre it jumps a few pixels. How would you fix that?

Record the offset on pointer down: `grabOffset = clientX - handleCenterX` if the pointer is on the handle, else 0. Then on move use `clientX - grabOffset`. The divider then stays exactly where it was grabbed and moves with the finger.

Q18. Should a click animate to the new position instead of jumping?

It can: add a `transition` on `clip-path` and `left` only while not dragging (a "dragging" class turns it off, or else every move lags behind the pointer). Respect `prefers-reduced-motion` and skip the animation there.

Q19. Why are the labels inside the layers?

The "Before" label lives in the clipped layer, so when the divider passes over it, it is clipped away with its image. The "After" label is under the before layer, so it is covered once the before image reaches it. No JS needed to hide them.

Performance, scale, quality

Q20. What if there are 50 sliders on a page (e.g. a product gallery)?

Each slider is independent with local state, so dragging one re-renders only that one. Lazy-load the images (`loading="lazy"`), use `srcSet` / `sizes` so phones don't download 4K photos, and render a slider only when the list item is in view if the images are heavy.

Q21. The images are 4000 px wide. What changes?

Serve responsive sizes (`srcSet`), add `decoding="async"` , and keep the layers on the GPU (`clip-path` + absolute layers already paint cheaply). Avoid `filter` or `box-shadow` on the big layers during drag, as they make repaints expensive.

Q22. How does this work in an RTL page?

The clip and math are physical (left/right), which is right for images: "before on the left" is a design choice, not text direction. If the product wants before on the start side in RTL, flip with `dir` : compute `100 - pct` and swap the inset sides and the arrow key mapping.

Q23. How would you test this?

Unit tests for the pure functions (clamp edges, pointer outside the rect, width 0, every key). React Testing Library: render, fire `keyDown` on `role="slider"` , assert `aria-valuenow` and the `--pos` style, assert `onChange` calls; fire `error` on an image and assert the alert. Playwright for the real drag, drag outside the frame, and a touch emulation scroll check.

Q24. How would you make it a reusable library component?

Keep the props contract small, add `value` (controlled), `orientation` , `mode` , `renderHandle` for a custom handle, forward a ref and `className` , and ship CSS with variables (`--ic-handle-size` , `--ic-divider-color`) so teams can theme it without overriding selectors.